Phase 0 Math: Worked Solutions

October 12, 2026

Contents

1 The toolkit: what you need before the problems **3**

1.1 1. What a derivative is . . . 3

1.2 2. The rules you need . . . 3

1.3 3. The chain rule . . . 3

1.4 4. Partial derivatives and the gradient . . . 4

1.5 5. Vectors and matrices . . . 4

2 Problem 1: The gradient of the squared error **4**

2.1 What this problem is really about . . . 5

2.2 Step by step . . . 5

2.3 Check it with real numbers . . . 5

2.4 Bonus: the best weights . . . 6

3 Problem 2: The derivative of the sigmoid **6**

3.1 What the sigmoid does . . . 6

3.2 Step by step . . . 6

3.3 Check it with real numbers . . . 7

3.4 Why this matters later . . . 7

4 Problem 3: Softmax plus cross-entropy **7**

4.1 What this problem is really about . . . 7

4.2 Step by step . . . 8

4.3 Check it with real numbers . . . 8

4.4 What the answer means . . . 9

4.5 Why it matters later . . . 9

5 Problem 4: The best estimate of a bell curve's center **9**
5.1 What this problem is really about . . . 9
5.2 The pieces . . . 9
5.3 Step by step . . . 9
5.4 Check it with real numbers . . . 10
5.5 Why it matters later . . . 10

6 Problem 5: What the SVD means geometrically **10**
6.1 The picture: a circle becomes an ellipse . . . 10
6.2 A worked 2×2 example . . . 11
6.3 What the singular values tell you . . . 11
6.4 Why it matters later . . . 11

7 How to check yourself **12**
7.1 Redo each problem from scratch . . . 12
7.2 Try new numbers . . . 12
7.3 Explain it out loud . . . 12
7.4 Where each problem shows up again . . . 12

1 The toolkit: what you need before the problems

This section teaches the five ideas the problems use: derivatives, the rules for computing them, the chain rule, partial derivatives and gradients, and vector and matrix notation. If you understand this page, every problem below is just these ideas applied carefully.

1.1 1. What a derivative is

A derivative is a **slope**: how fast a function's output changes when you nudge its input a tiny bit. For f(x), the derivative is written f′(x) or df/dx.

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

In plain words: move x by a tiny amount h, see how much f changed, and divide by h. As h shrinks toward zero, that ratio is the slope at x.

Worked example. Let f(x) = x^2 at x = 3. Try h = 0.001: f(3.001) = 9.006001, f(3) = 9, so the slope is about 0.006001 / 0.001 = 6.001. The exact answer is 6, and the leftover 0.001 disappears as h shrinks.

Why ML cares: training a model means asking "if I nudge this weight, does the error go up or down, and how fast?" That question is a derivative.

1.2 2. The rules you need

Rule	Function	Derivative
Constant	c	0
Power	x^n	$n\,x^{n-1}$
Constant multiple	c · f(x)	c · f′(x)
Sum	f(x) + g(x)	f′(x) + g′(x)
Exponential	e^x	e^x
Logarithm	ln x	1 / x
Product	f(x) · g(x)	f′g + fg′
Quotient	f(x) / g(x)	(f′g − fg′) / g^2

Examples: the derivative of $3x^4$ is $12x^3$. The derivative of $x^2 + 5x$ is 2x + 5. The derivative of 1/x (which is x^{-1}) is $-x^{-2} = -1/x^2$.

1.3 3. The chain rule

If y = f(g(x)), then:

$$\frac{dy}{dx} = f'(g(x)) \cdot g'(x)$$

In plain words: **the derivative of the outside, evaluated at the inside, times the derivative of the inside.**

Worked example. $y = (3x + 1)^2$. The outside is u^2 (derivative $2u$); the inside is $u = 3x + 1$ (derivative 3). So $dy/dx = 2(3x + 1) \cdot 3 = 6(3x + 1)$. At $x = 1$, that is 24.

Why ML cares: a neural network is functions inside functions inside functions. Backpropagation, which you build in Phase 3, is the chain rule applied over and over, from the error back to every weight.

1.4 4. Partial derivatives and the gradient

When a function has several inputs, like $f(x, y) = x^2y + 3y$, a **partial derivative** asks how f changes when you nudge just one input and freeze the others.

- $\partial f/\partial x$: treat y as a constant. You get $2xy$.
- $\partial f/\partial y$: treat x as a constant. You get $x^2 + 3$.

The **gradient** collects all the partial derivatives into one vector, written ∇f:

$$\nabla f = \begin{bmatrix} \partial f/\partial x \\ \partial f/\partial y \end{bmatrix} = \begin{bmatrix} 2xy \\ x^2 + 3 \end{bmatrix}$$

The gradient points in the direction where f increases fastest. **Gradient descent** steps the opposite way to make f smaller. That's how every model in this project learns.

1.5 5. Vectors and matrices

- A **vector** is a list of numbers, written as a column.
- A **matrix** is a grid of numbers. A matrix with m rows and n columns has shape $m \times n$.
- **Transpose** (^{T}) flips rows and columns. A 2×3 matrix becomes 3×2.
- **Matrix–vector product** Ax: each entry of the result is a row of A multiplied entry by entry with x, then added up.
- **Dot product** $u^Tv = u_1v_1 + u_2v_2 + \ldots$, a single number.
- **Squared length** $\|v\|^2 = v^Tv = v_1^2 + v_2^2 + \ldots$

Worked example.

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad x = \begin{bmatrix} 5 \\ 6 \end{bmatrix}, \quad Ax = \begin{bmatrix} 1 \cdot 5 + 2 \cdot 6 \\ 3 \cdot 5 + 4 \cdot 6 \end{bmatrix} = \begin{bmatrix} 17 \\ 39 \end{bmatrix}$$

And $\|x\|^2 = 5^2 + 6^2 = 61$.

2 Problem 1: The gradient of the squared error

Task: derive the gradient of $f(x) = \|Ax - b\|^2$ with respect to x.

Answer:

$$\nabla_x f = 2A^\top(Ax - b)$$

2.1 What this problem is really about

This is **linear regression**. A is your data (one row per example, one column per feature), x is the weights, and b is the correct answers. Ax is your predictions, Ax − b is how wrong each one is, and $\|Ax - b\|^2$ adds up the squared errors. The gradient tells you how to change the weights to shrink that total error.

2.2 Step by step

Step 1: Name the errors. Let r = Ax − b, the **residual** vector. Then $f = \|r\|^2 = r_1^2 + r_2^2 + \ldots + r_m^2$.

Step 2: Write one error in full.

$$r_i = A_{i1}x_1 + A_{i2}x_2 + \cdots + A_{in}x_n - b_i$$

Step 3: How does one error change when we nudge x_j? Every term without x_j is a constant, so only $A_{ij}x_j$ remains:

$$\frac{\partial r_i}{\partial x_j} = A_{ij}$$

Step 4: Apply the chain rule to the whole sum.

$$\frac{\partial f}{\partial x_j} = \sum_{i=1}^{m} 2r_i \frac{\partial r_i}{\partial x_j} = \sum_{i=1}^{m} 2r_i A_{ij}$$

Step 5: Recognize the matrix form. The sum over i of $A_{ij}r_i$ is entry j of A^Tr. Stacking all entries gives $\nabla f = 2A^Tr = 2A^T(Ax - b)$.

Shape check: A is m × n, so A^T is n × m; A^Tr has n entries, one per weight. Correct.

2.3 Check it with real numbers

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad x = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

1. Predictions: Ax = [1, 3].
2. Errors: r = Ax − b = [0, 2].
3. Total error: $f = 0^2 + 2^2 = 4$.
4. $A^Tr = [1\cdot 0 + 3\cdot 2,\ 2\cdot 0 + 4\cdot 2] = [6, 8]$.
5. Gradient: $2A^Tr =$ **[12, 16]**.

Numerical check. Nudge x_1 to 1.001: r = [0.001, 2.003], f = 4.01201. Change 0.01201 / 0.001 = **12.01**, matching 12. Your Phase 3 `gradcheck` does exactly this.

2.4 Bonus: the best weights

Setting the gradient to zero gives the **normal equations**, the closed-form solution you'll implement in Phase 6:

$$A^\top A\, x = A^\top b$$

3 Problem 2: The derivative of the sigmoid

Task: show that the derivative of the sigmoid equals $\sigma(x)(1 - \sigma(x))$.

$$\sigma(x) = \frac{1}{1 + e^{-x}}, \qquad \sigma'(x) = \sigma(x)\left(1 - \sigma(x)\right)$$

3.1 What the sigmoid does

It squashes any number into 0 to 1, turning a score into a probability. Logistic regression (Phase 6) and LSTM gates (Phase 7) use it.

x	e^{-x}	$\sigma(x)$
−4	54.598	0.018
0	1	0.500
2	0.135	0.881
4	0.018	0.982

3.2 Step by step

Step 1: Rewrite it as a power.

$$\sigma(x) = \left(1 + e^{-x}\right)^{-1}$$

Step 2: Chain rule. The outside u^{-1} has derivative $-u^{-2}$; the inside $1 + e^{-x}$ has derivative $-e^{-x}$:

$$\sigma'(x) = -\left(1 + e^{-x}\right)^{-2} \cdot \left(-e^{-x}\right) = \frac{e^{-x}}{\left(1 + e^{-x}\right)^2}$$

Step 3: Split into two pieces.

$$\sigma'(x) = \frac{1}{1+e^{-x}} \cdot \frac{e^{-x}}{1+e^{-x}}$$

The first piece is σ(x).

Step 4: The second piece is 1 − σ(x).

$$1 - \sigma(x) = \frac{1+e^{-x}}{1+e^{-x}} - \frac{1}{1+e^{-x}} = \frac{e^{-x}}{1+e^{-x}}$$

So σ′(x) = σ(x)(1 − σ(x)).

3.3 Check it with real numbers

- **x = 0:** 0.5 × 0.5 = **0.25**, the steepest the sigmoid gets.
- **x = 2:** 0.8808 × 0.1192 = **0.1050**; Step 2's form gives 0.1353 / 1.2889 = 0.1050. Both agree.
- **x = 4:** about **0.018**, nearly flat.

3.4 Why this matters later

1. **Speed in backprop.** The forward pass already computes y = σ(x), so the backward pass is just y(1 − y).
2. **Vanishing gradients.** The slope is never above 0.25, so stacked sigmoid layers multiply small numbers and early layers stop learning. That's one reason modern networks use ReLU (Phase 7).

4 Problem 3: Softmax plus cross-entropy

Task: show that the gradient of softmax followed by cross-entropy, with respect to the logits, equals p − y.

$$\frac{\partial L}{\partial z} = p - y$$

where p is the predicted probabilities and y is the one-hot correct answer.

4.1 What this problem is really about

This is how **every classifier** in the project measures its error. The model outputs raw scores, called **logits** (z).

1. **Softmax** turns scores into probabilities that add up to 1:

$$p_i = \frac{e^{z_i}}{\sum_k e^{z_k}}$$

2. **Cross-entropy** measures how much probability went to the correct class c:

$$L = -\sum_i y_i \log p_i = -\log p_c$$

4.2 Step by step

Step 1: Combine them.

$$L = -\log \frac{e^{z_c}}{\sum_k e^{z_k}} = -z_c + \log \sum_k e^{z_k}$$

The second term is **log-sum-exp**, which you'll implement stably in Phase 2.

Step 2: the derivative of $-z_c$ with respect to z_j is -1 if j is the correct class, else 0. That is $-y_j$.

Step 3: the derivative of log-sum-exp:

$$\frac{\partial}{\partial z_j} \log \sum_k e^{z_k} = \frac{e^{z_j}}{\sum_k e^{z_k}} = p_j$$

Step 4: Add them.

$$\frac{\partial L}{\partial z_j} = p_j - y_j$$

4.3 Check it with real numbers

Logits z = [2, 1, 0], correct class first, so y = [1, 0, 0].

1. $e^2 = 7.389$, $e^1 = 2.718$, $e^0 = 1$. Sum = 11.107.
2. p = **[0.665, 0.245, 0.090]**.
3. L = $-\log(0.665)$ = **0.408**.
4. Gradient p − y = **[−0.335, 0.245, 0.090]**.

Numerical check. Nudge z_1 by 0.001: $-z_c$ drops 0.001, log-sum-exp rises 0.000665. Net −0.000335 / 0.001 = **−0.335**. Matches.

4.4 What the answer means

- The correct class's score goes up, more when the model was less confident.
- Every wrong class's score goes down, in proportion to the probability it wrongly got.
- A perfect prediction (p = y) gives zero gradient.
- The gradient always sums to zero: probability is only moved between classes.

4.5 Why it matters later

Softmax's own derivative is messy, but combined with cross-entropy it collapses to p − y. That's why frameworks, including yours in Phase 4, fuse them into one `cross_entropy(logits, targets)` function.

5 Problem 4: The best estimate of a bell curve's center

Task: show that the maximum likelihood estimate (MLE) of a Gaussian's mean is the sample mean.

$$\hat{\mu} = \frac{1}{n}\sum_{i=1}^{n} x_i$$

5.1 What this problem is really about

Maximum likelihood asks: which μ makes the data you saw most probable? It's the idea behind nearly every loss function in ML.

5.2 The pieces

$$p(x \mid \mu) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

For independent points, the **likelihood** is the product:

$$\mathcal{L}(\mu) = \prod_{i=1}^{n} p(x_i \mid \mu)$$

5.3 Step by step

Step 1: Take the log. Logs turn products into sums, and the same μ maximizes both:

$$\ell(\mu) = -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}(x_i - \mu)^2$$

Step 2: Differentiate with respect to μ.

$$\frac{d\ell}{d\mu} = \frac{1}{\sigma^2}\sum_{i=1}^{n}(x_i - \mu)$$

Step 3: Set it to zero.

$$\sum_{i=1}^{n}(x_i - \mu) = 0 \;\Rightarrow\; \mu = \frac{1}{n}\sum_{i=1}^{n} x_i$$

Step 4: It's a maximum, because the second derivative $-n/\sigma^2$ is negative.

5.4 Check it with real numbers

Data [2, 4, 9], average **5**. Maximizing likelihood = minimizing $\Sigma(x_i - \mu)^2$:

Guess μ	Squared distances	Total
4	4 + 0 + 25	29
5	**9 + 1 + 16**	**26**
6	16 + 4 + 9	29

5.5 Why it matters later

Maximizing a Gaussian likelihood is exactly **minimizing squared error**, which is why MSE is the standard regression loss. Cross-entropy (Problem 3) is maximum likelihood for class labels.

6 Problem 5: What the SVD means geometrically

Task: explain, with a 2×2 example, what the SVD tells you geometrically.

Answer: every matrix does three things, in order: **rotate, stretch along the axes, rotate again.**

$$A = U\,\Sigma\,V^{\top}$$

- $\mathbf{V^s}$ rotates (or flips) the input.
- **Σ** is diagonal: it stretches along each axis by the **singular values** $\sigma_1 \geq \sigma_2 \geq 0$.
- **U** rotates (or flips) the result.

6.1 The picture: a circle becomes an ellipse

Multiply every point on a unit circle by A and you get an **ellipse**: longest radius σ_1 along U's first column, shortest σ_2 along U's second column. Draw it in your notebook.

6.2 A worked 2×2 example

$$A = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}$$

Step 1: Singular values are square roots of the eigenvalues of A^TA:

$$A^\top A = \begin{bmatrix} 10 & 6 \\ 6 & 10 \end{bmatrix}$$

$(10 - \lambda)^2 - 36 = 0$ gives $\lambda = 16$ or 4, so **$\sigma_1 = 4$, $\sigma_2 = 2$**.

Step 2: Input directions (V) are eigenvectors of A^TA: (1, 1) for $\lambda = 16$ and (1, −1) for $\lambda = 4$, each divided by $\sqrt{2}$.

Step 3: Where they land (U).

$$A \begin{bmatrix} 1 \\ 1 \end{bmatrix} = 4 \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \qquad A \begin{bmatrix} 1 \\ -1 \end{bmatrix} = 2 \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

Neither changes direction, so U = V here, because A is symmetric.

Step 4: Put it together.

$$A = \tfrac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 4 & 0 \\ 0 & 2 \end{bmatrix} \tfrac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

Multiply it out to confirm you get [[3, 1], [1, 3]].

Geometrically: A turns the unit circle into an ellipse tilted 45°, radius 4 along (1, 1) and radius 2 along (1, −1).

6.3 What the singular values tell you

What you see	What it means
All σ equal	Circles stay circles
σ_1 much bigger than σ_2	Space is squashed into nearly one direction
Some σ = 0	A flattens a direction; it can't be inverted
Number of nonzero σ	The **rank**
σ_1 / σ_smallest	The **condition number**

6.4 Why it matters later

- **PCA (Phase 6):** the SVD finds the directions where data varies most.
- **LoRA (Phase 13):** a low-rank update instead of a full matrix.
- **Optimization (Phase 16):** the condition number controls convergence speed.

7 How to check yourself

7.1 Redo each problem from scratch

Close this document and work each problem on a blank page. Tick it only when you can do it without looking.

- ☐ **Toolkit:** differentiate $(3x + 1)^2$, and find the gradient of $x^2y + 3y$.
- ☐ **Problem 1:** derive $2A^T(Ax - b)$ and get [12, 16].
- ☐ **Problem 2:** derive $\sigma(x)(1 - \sigma(x))$ and compute the slope at 0.
- ☐ **Problem 3:** derive $p - y$ and compute it for $z = [2, 1, 0]$.
- ☐ **Problem 4:** derive the MLE and explain why it gives squared error.
- ☐ **Problem 5:** compute the singular values of [[3, 1], [1, 3]] and draw the ellipse.

7.2 Try new numbers

1. Problem 1 with $x = [0, 1]$.
2. Problem 2 at $x = -2$. (Compare it to $x = 2$.)
3. Problem 3 with $z = [0, 0, 0]$.
4. Problem 4 with data [1, 1, 10]. Is the average a good center?
5. Problem 5 with $A = [[2, 0], [0, 1]]$.

7.3 Explain it out loud

For each problem, explain in under a minute, without notes, what it computes and why ML needs it.

7.4 Where each problem shows up again

Problem	Used again in
Chain rule	Phase 3 autograd
1. Squared-error gradient	Phase 6 linear regression and Ridge
2. Sigmoid derivative	Phases 3, 6 and 7
3. Softmax + cross-entropy	Phase 4 losses, every classifier, the GPT
4. Maximum likelihood	Phases 4, 9 and 18
5. SVD	Phases 6, 13 and 16